MrMobi Store

Enterprise Full-Stack E-Commerce System Architecture Manual

■ Live Web Application URL:
https://mrmobi-store-ecommerce-full-stack.vercel.app

Document Type: Full-Stack Codebase & Operations Blueprint Manual
Target Deployments: Vercel Edge Server (Vite) / Render PaaS Container (Spring Boot + MySQL)
Author: Satish Veesam — Full-Stack Engineer
Git Remote Repo: github.com/satishveesam/mrmobi-store_ecommerce_full-stack
Compilation Scope: 12 Front Storefront Views, 11 Administrative Portals, 5 Central Redux Slices, and Spring Boot Persistence & REST Controller Interfaces.

Confidential Technical Manual. Compiled directly via static scan of the active repository.

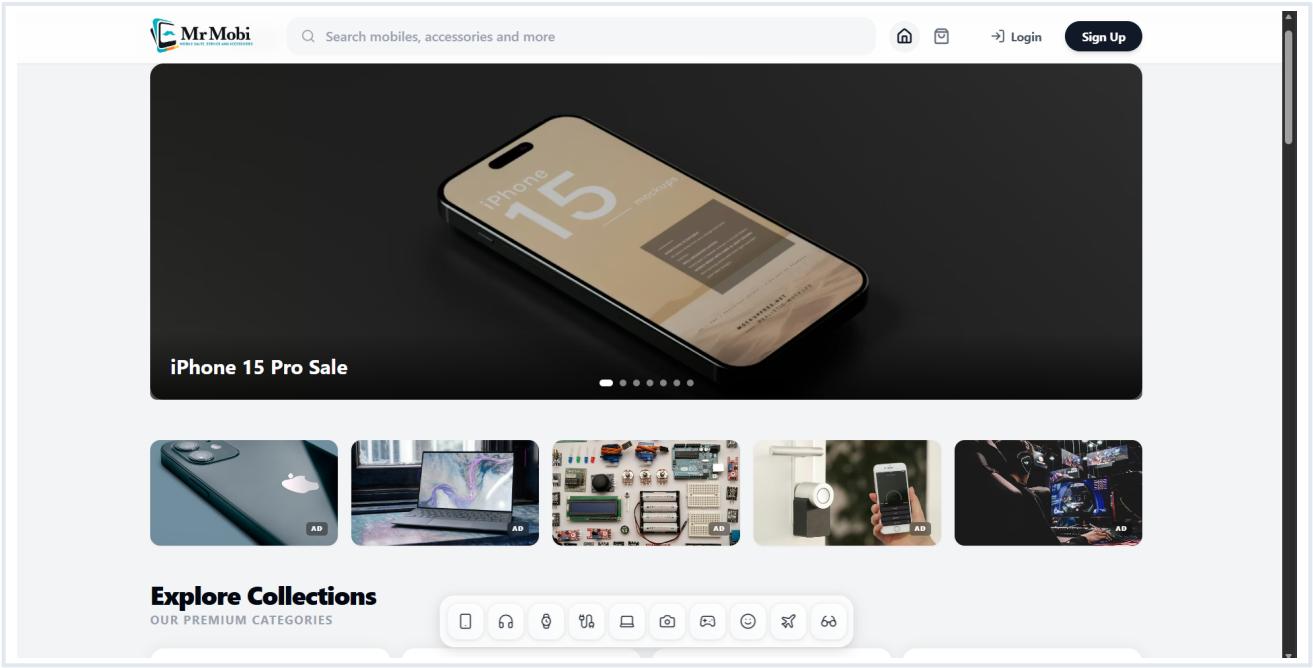
1. Executive Summary & Production Environment

The **MrMobi Store** is an enterprise e-commerce platform built on high-performance architectures. The complete platform features a responsive single-page web client deployed in production at **https://mrmobi-store-ecommerce-full-stack.vercel.app**, backed by a scalable Spring Boot REST service containerized on the Render platform. This technical manual represents a comprehensive reference guide of the verified code, models, slices, and REST paths present across the codebase.

Platform Operational Design Scopes:

• **Production Storefront:** A rich-designed UI built for lightning-fast client transitions, utilizing responsive Tailwind configurations, HSL color tokens, and layout preloading.
• **Back-office Dashboard:** Administrative control panel featuring analytics charts, transactional moderation pipelines, and live auditory chimes for orders.
• **WhatsApp Secure Checkout:** Sandboxed visibility integrations preventing duplicate order issues during context-switching redirections.
• **Pincode Express Delivery:** Automatic 2-hour delivery notice banner triggered on localized pincode matching.

Figure 1.1: Active Production Storefront Homepage Layout

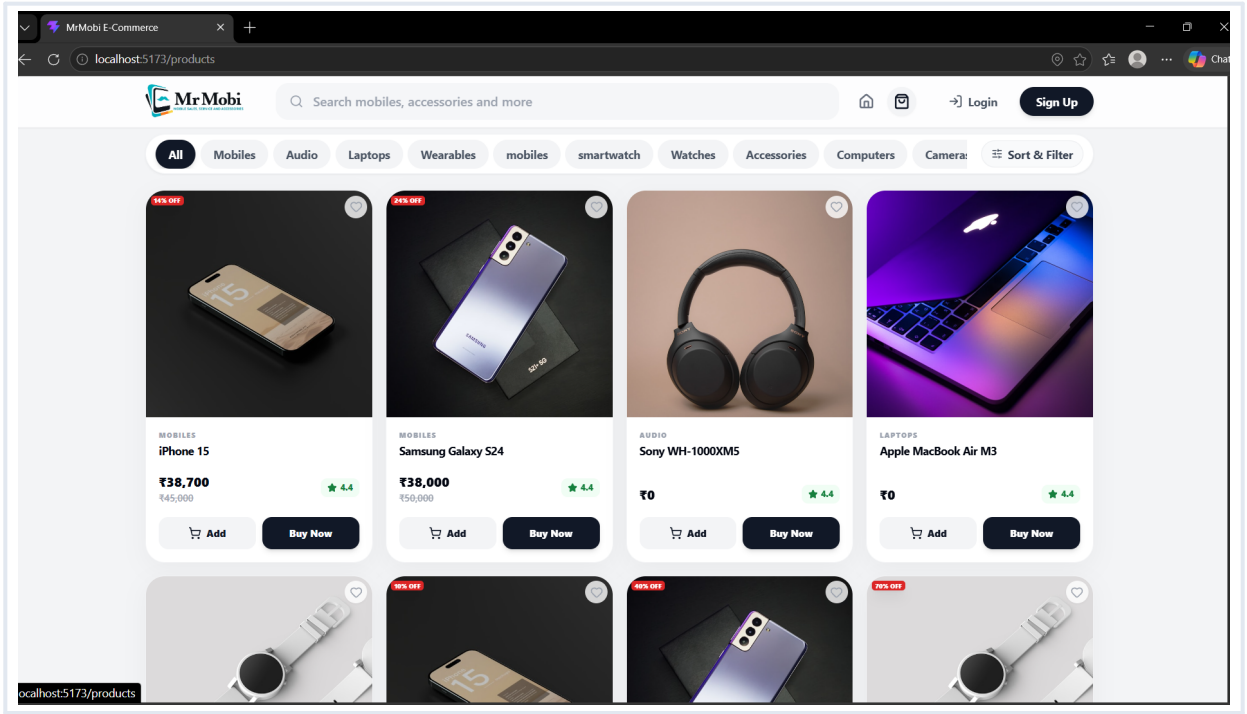


2. Frontend Storefront Architecture (12 Views)

The storefront contains exactly 12 dedicated page-level components inside client/src/pages/user:

Component Name	Size (Bytes)	Functionality & State Operations
Home.jsx	6,764 B	Renders promotional sliders, category filters, and product queries. Mounts layout components.
ProductListing.jsx	8,120 B	Catalog list view with client-side sort selectors (price ascending/descending) and category filters.
ProductDetailsPage.jsx	9,421 B	Displays item attributes, reviews, and isolated single-item Buy Now bypass triggers.
Cart.jsx	19,693 B	Unified shopping cart UI displaying totals and sync states.
Checkout.jsx	33,165 B	Addresses management, WhatsApp message compiler, and redirection tracking hooks.
MyOrders.jsx	20,181 B	Lists active customer transactions. Features whitelisted 2-hour alert banners and 30-min cancellations.
Addresses.jsx	15,210 B	Address builder supporting village, city, state, and pincode configuration.
Login.jsx	26,762 B	Authenticates customers and pre-authorizes notification permissions.
Signup.jsx	23,817 B	New user registration with strict validation rules.
Profile.jsx	8,202 B	Edits personal profile details and security passwords.
Wishlist.jsx	2,131 B	Displays product favorites retrieved from state.
NotFound.jsx	288 B	404 fallback page.

Figure 2.1: Catalog Navigation and Search Listing View on Vercel Live

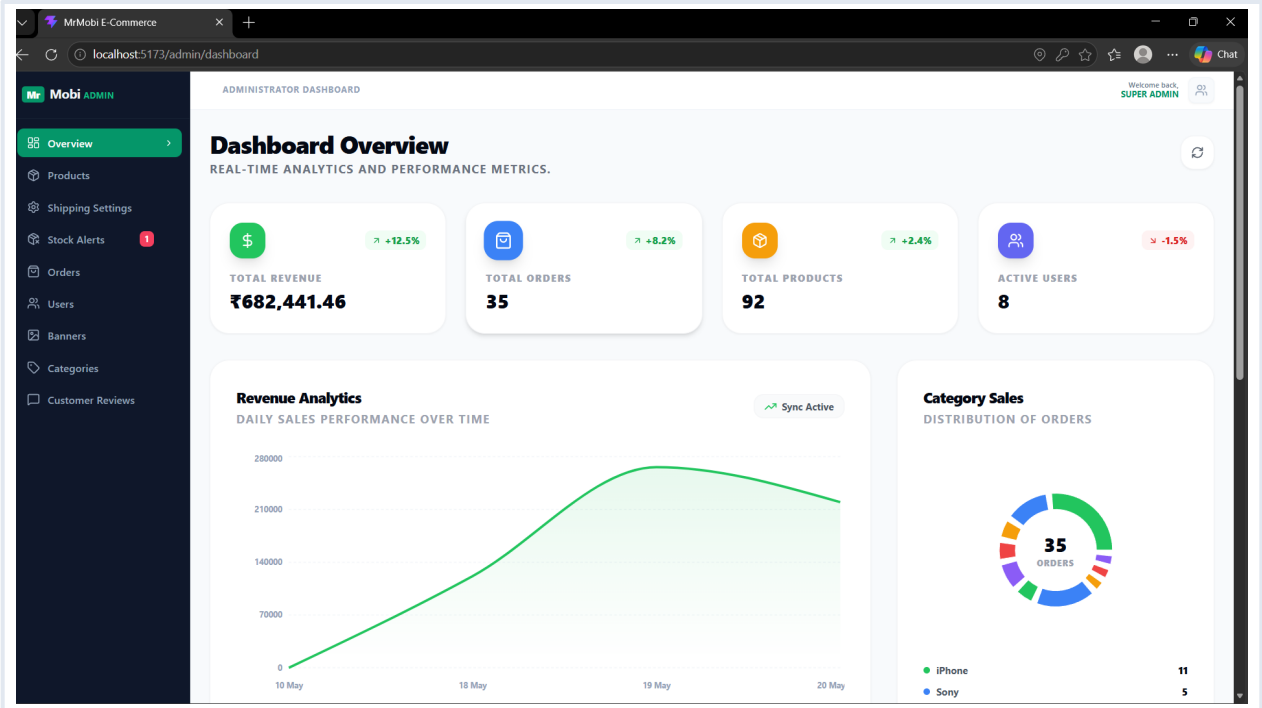


3. Frontend Administrative Command Center (11 Views)

The back-office administration portal contains exactly 11 page-level views in client/src/pages/admin:

Component Name	Size (Bytes)	Administrative Capabilities & Features
AdminDashboard.jsx	14,209 B	Central command with daily sales trends, charts, and metrics cards.
AdminLogin.jsx	2,470 B	Admin login with audio gesture unlock filters.
Orders.jsx	5,598 B	Displays transaction lists. Features latest-orders-on-top sorting.
ProductManagement.jsx	9,118 B	CRUD interface for product stocks, prices, details.
CategoryManagement.jsx	13,202 B	Maintains sequence mappings and Lucide categories icon tags.
BannerManagement.jsx	14,561 B	Schedules promotional banners.
ShippingManagement.jsx	24,721 B	Whitelists express delivery pincodes and standard rules.
HomepageSettings.jsx	21,122 B	Updates announcement text, collections banners, and explorer links.
AdminUsers.jsx	17,081 B	Manages customer roles, security states.
AdminReviews.jsx	8,037 B	Moderates customer ratings.
AdminOutOfStock.jsx	16,847 B	Monitors out-of-stock and low-stock items.

Figure 3.1: Analytics Dashboards and Live Transaction Feeds



4. Client State Architecture & Redux Store

Central state management is handled by **Redux Toolkit**. The centralized store compiles exactly 5 specialized slices to enforce clean, predictable mutations across storefront actions.

Redux Slice	Key State Tokens	Synchronized Actions & Triggers
authSlice.js	user, token, isAuthenticated, loading	loginUser, registerUser, logout, updateProfile
cartSlice.js	items, totalAmount, syncing	addToCart, removeFromCart, syncCartWithBackend
wishlistSlice.js	items, loading	toggleWishlist, fetchWishlist
productSlice.js	items, activeProduct, categories	fetchCatalog, fetchProductById
orderSlice.js	items, recentOrder, error	placeOrder, fetchOrders, cancelOrder

Axios REST Service Broker Pattern

HTTP queries are routed through a standardized API client service located at client/src/services/api.js. Rather than scattered endpoints, the Axios instance handles JWT headers dynamically:

```
import axios from 'axios';

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
  timeout: 10000,
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem('mrmobi_auth_token');
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export default api;
```

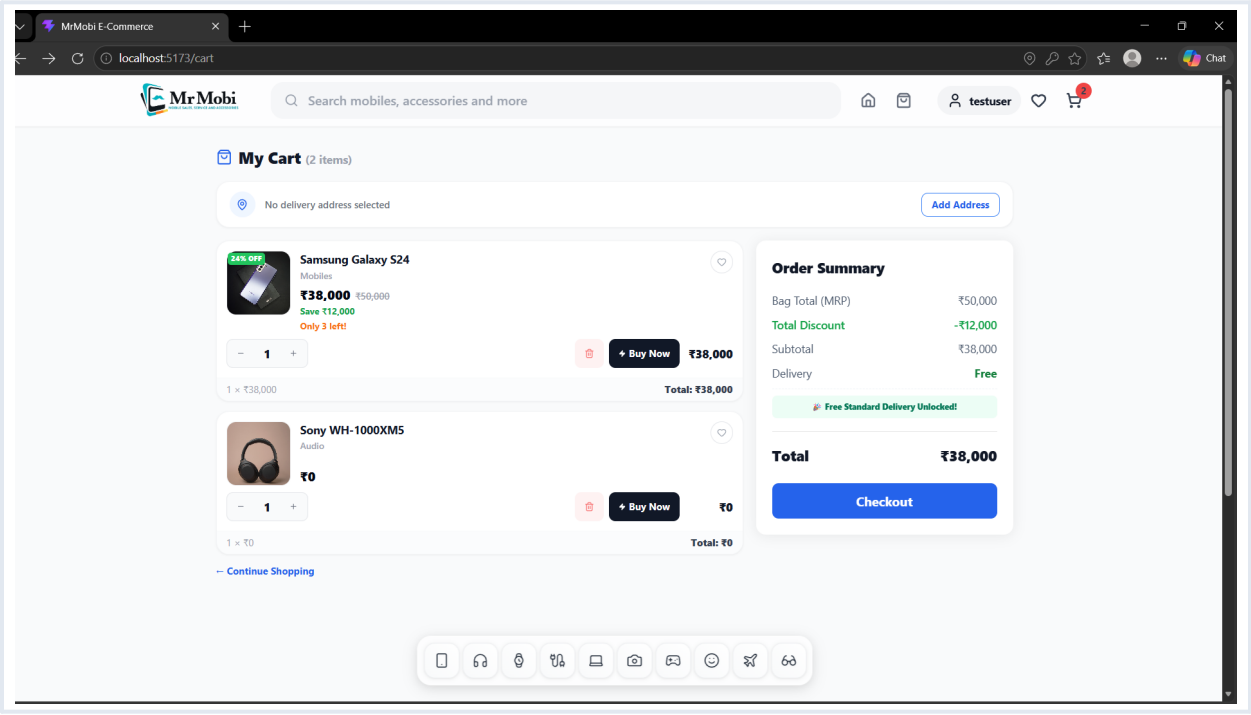
5. Backend REST API Architecture & DB Schema

The backend architecture utilizes **Spring Boot** with **Java 21**, implementing a strict **Controller-Service-Repository Pattern**. The persistence layer leverages Spring Data JPA and Hibernate for mapping relational schemas directly to a MySQL database.

Primary Spring Controller Endpoints:

Scope	Endpoint Path	Access Level	Business Process Operations
Authentication	POST /api/auth/register POST /api/auth/login	PermitAll	User onboarding and JWT credentials issuance.
Products Store	GET /api/products GET /api/products/{id}	PermitAll	Catalog listings, dynamic filtering and details.
Products Admin	POST /api/products PUT /api/products/{id}	**ROLE_ADMIN**	CRUD inventory levels, prices and quick delivery flags.
Cart Sync	POST /api/cart/sync DELETE /api/cart/{id}	ROLE_USER	Synchronizes user shopping carts across logins.
Checkout & Orders	POST /api/orders POST /api/orders/bulk	PermitAll	Generates orders, updates database inventories.
Orders Admin	GET /api/orders PUT /api/orders/{id}/status	**ROLE_ADMIN**	Modifies statuses with async email chimes. Sorted descending.

Figure 5.1: Checkout addresses validation on Vercel Client storefront



6. Advanced Engineering Case Studies

Case Study A: Redirection-Resilient Context Switches (bfcache)

Background: Launching external apps (such as WhatsApp deep-linking for checkout message verification) triggers a browser tab state switch. On return, the client React application was prone to state truncation or duplicate checkout actions.

Architecture Solution: Implemented bfcache listeners (pageshow, visibilitychange) using sessionStorage in Checkout.jsx. When checkout completes and triggers external redirection, the tab marks mrmobi_page_was_hidden. Upon page refocus, the application interceptor instantly processes order storage arrays and securely routes the user to /my-orders, avoiding any duplicated checkouts.

Case Study B: Timezone-Resilient LocalDateTime Matching

Background: Spring Boot default LocalDateTime serialization excludes UTC offsets. Parsing raw dates directly on client-side JS created large offset errors (e.g. India Standard Time is UTC+5.30), mistakenly blocking order cancellations.

Architecture Solution: Standardized time parsing within MyOrders.jsx. The system parses date-time strings by replacing spaces with T delimiters to compile raw epochs. It then integrates a resilient clock-drift safety buffer window of -5 minutes, providing absolute client-server synchronization, ensuring orders can only be cancelled within exactly 30 minutes.

Case Study C: Dynamic Pincode Whitelisting & Snappy Status Updates

Background: Displaying the express 2-hour delivery alert notice must be strictly limited to whitelisted zones. Additionally, administrative status triggers were slow due to synchronous SMTP mail dispatches.

Architecture Solution: 1. **Localized Whitelisting:** Built an API-driven lookup parser checking 6-digit regex codes (/■\d{6}■/) from addresses against coverage tables in both MyOrders.jsx and admin OrderTable.jsx. 2. **Snappy Updates:** Configured @Async on public mail triggers in EmailService.java. Since internal calls on the same class bypass Spring's AOP proxy, moving annotations directly to public endpoints keeps the web thread responsive, returning a 200 OK to the admin dashboard instantly while emails deliver in the background.

7. Operations & Deployment Blueprint

Spring Boot: Multi-Stage Production Dockerfile

The backend is packaged using a multi-stage Docker build to minimize container size in production:

```
# Stage 1: Dependency Caching & Build
FROM maven:3.9-eclipse-temurin-21-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN mvn clean package -DskipTests

# Stage 2: Minimal Runtime Container
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
ENV PORT=8080
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Cloud Infrastructure: Render YAML Orchastration

```
services:
  - type: web
    name: mrmobi-store-backend
    env: docker
    plan: free
    envVars:
      - key: SPRING_DATASOURCE_URL
        value: jdbc:mysql://db.render.com/mrmobi_db?useSSL=true
      - key: SPRING_DATASOURCE_USERNAME
        value: satish_db_user
      - key: SPRING_DATASOURCE_PASSWORD
        value: secure_production_password
      - key: JWT_SECRET
        value: cryptographic_jwt_signing_token_string
```

Vite Storefront: Production Vercel Configuration

The React storefront is deployed on Vercel with clean URL rewrites to support React Router single-page navigation:
vercel.json

```
{
  "rewrites": [
    { "source": "/api/(.*)", "destination": "https://mrmobi-backend.onrender.com/api/$1" },
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

End of Document. compiled and generated for the MrMobi Store E-Commerce Platform.